

Integration functions

```
import joblib

import numpy as np

from collections import Counter

def load_natiqi_model(model_path="natiqi_5word_svm_pipeline.pkl"):

    model = joblib.load(model_path)

    return model

def predict_windows(model, X_windows):

    """

    X_windows: numpy array of shape (n_windows, n_features)

    returns:

        pred_labels: window-level predicted class indices

        pred_proba: window-level probabilities

    """

    pred_labels = model.predict(X_windows)

    pred_proba = model.predict_proba(X_windows)

    return pred_labels, pred_proba

def predict_trial_majority_vote(model, X_windows):

    """

    Returns one prediction for the whole trial using majority vote.

    """

    pred_labels, _ = predict_windows(model, X_windows)

    final_pred = Counter(pred_labels.tolist()).most_common(1)[0][0]

    return int(final_pred)

def predict_trial_soft_vote(model, X_windows):

    """

    Returns one prediction for the whole trial using mean probabilities across windows.

    This is the method you selected as the final decision strategy.

    """

    _, pred_proba = predict_windows(model, X_windows)

    mean_proba = pred_proba.mean(axis=0)

    final_pred = int(np.argmax(mean_proba))

    confidence = float(np.max(mean_proba))

    return final_pred, confidence, mean_proba
```

Example to use

```
# load model

model = load_natqi_model("natqi_5word_svm_pipeline.pkl")

# Example:

# X_some_trial must be shape (n_windows, n_features)

pred_class, confidence, mean_proba = predict_trial_soft_vote(model,
X_some_trial)

print("Predicted class index:", pred_class)

print("Confidence:", confidence)

print("Mean probabilities:", mean_proba)
```

Convert the class to a word

```
label_map = {

    0: "جوع",

    1: "عطش",

    2: "انذار",

    3: "حمام",

    4: "نواء"

}

pred_word = label_map[pred_class]

print("Predicted word:", pred_word)
```